Airport Runway Scheduling Simulation

1. Scenario Selection & Problem Understanding

The project simulates a for landing and real-world airport runway management system, where multiple airplanes request access takeoff, and a central controller schedules runway usage safely.

This scenario is:

- Realistic & industry-relevant (Air Traffic Control Systems)
- Reflects concurrent resource management problems
- Naturally requires:
    - Multi-process coordination
    - IPC mechanisms
    - Threads for real-time scheduling
    - Signals for emergency handling

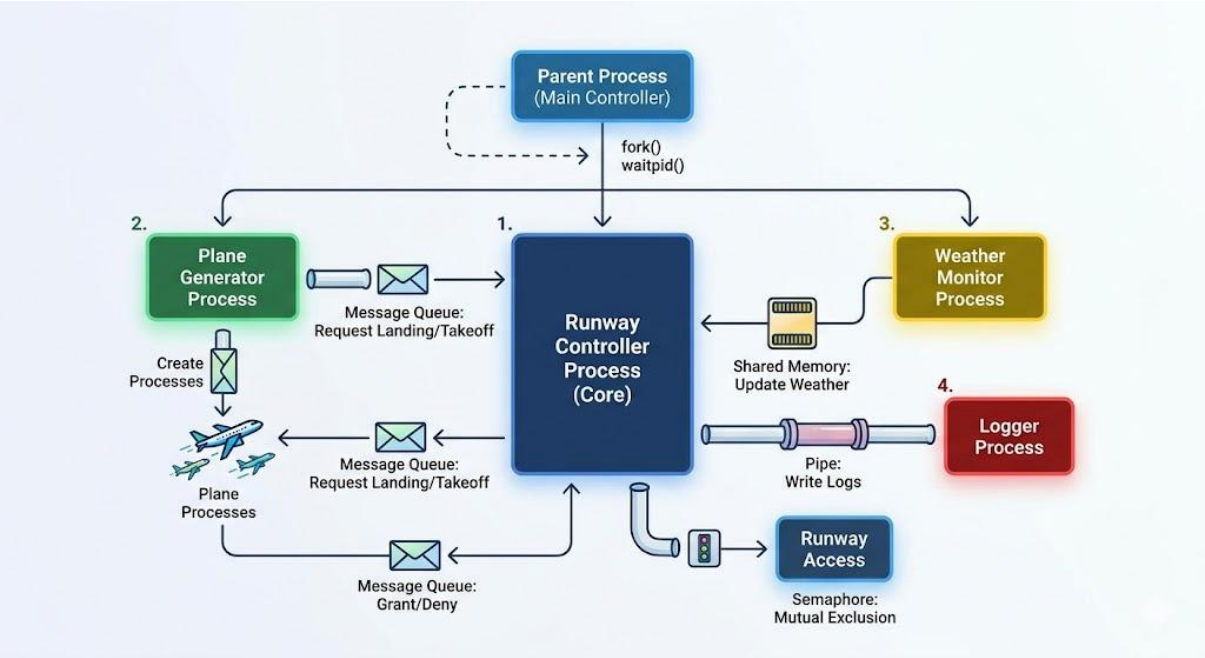
Problem Statement

Airports must ensure:

- Only one aircraft uses the runway at a time
- Emergency landings are prioritized
- Weather conditions affect scheduling
- System must handle multiple concurrent requests safely

Why Linux Concepts Are Needed

Feature	Justification
Processes	Each plane and subsystem is independent
IPC	Communication between planes and controller
Threads	Concurrent scheduling + monitoring
Signals	Emergency landing & shutdown
File I/O	Persistent logging



2. Process-Level Design

Architecture Overview

```
Parent Process (Main Controller)
|
 ├── Runway Controller Process
 ├── Plane Generator Process
 ├── Weather Monitor Process
 ├── Logger Process
```

Process Roles

1. Parent Process

- Initializes system
- Creates child processes using fork()
- Monitors using waitpid()
- Handles shutdown signals

2. Runway Controller Process (Core)

- Receives plane requests
- Maintains scheduling queue
- Grants runway access
- Ensures mutual exclusion

3. Plane Generator Process

- Generates planes randomly
- Sends requests via message queue

4. Plane Processes

- Represent real airplanes
- Wait for runway permission
- Perform landing/takeoff

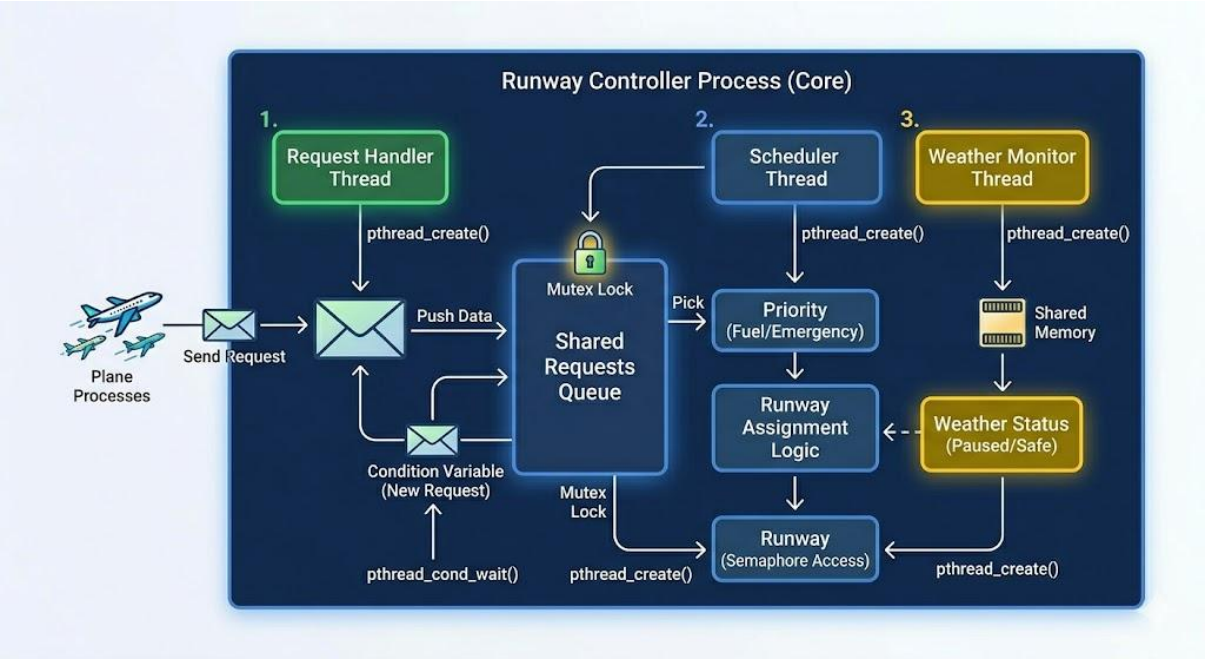
5. Weather Monitor Process

- Updates weather using shared memory

- Affects scheduling decisions

6. Logger Process

- Writes logs to file via pipe



3. Thread-Level Design

Threads are implemented inside Runway Controller

Threads Used

1. Request Handler Thread

- Continuously receives plane requests
- Pushes them into queue

2. Scheduler Thread

- Picks next plane

- Applies priority (fuel/emergency)
- Assigns runway

3. Weather Monitor Thread

- Reads shared memory
- Pauses scheduling if unsafe

Synchronization

- **Mutex**
  - Protects shared queue
- **Condition Variable**
  - Signals scheduler when new request arrives

4. Data Flow Diagram & Documentation

Process Flow

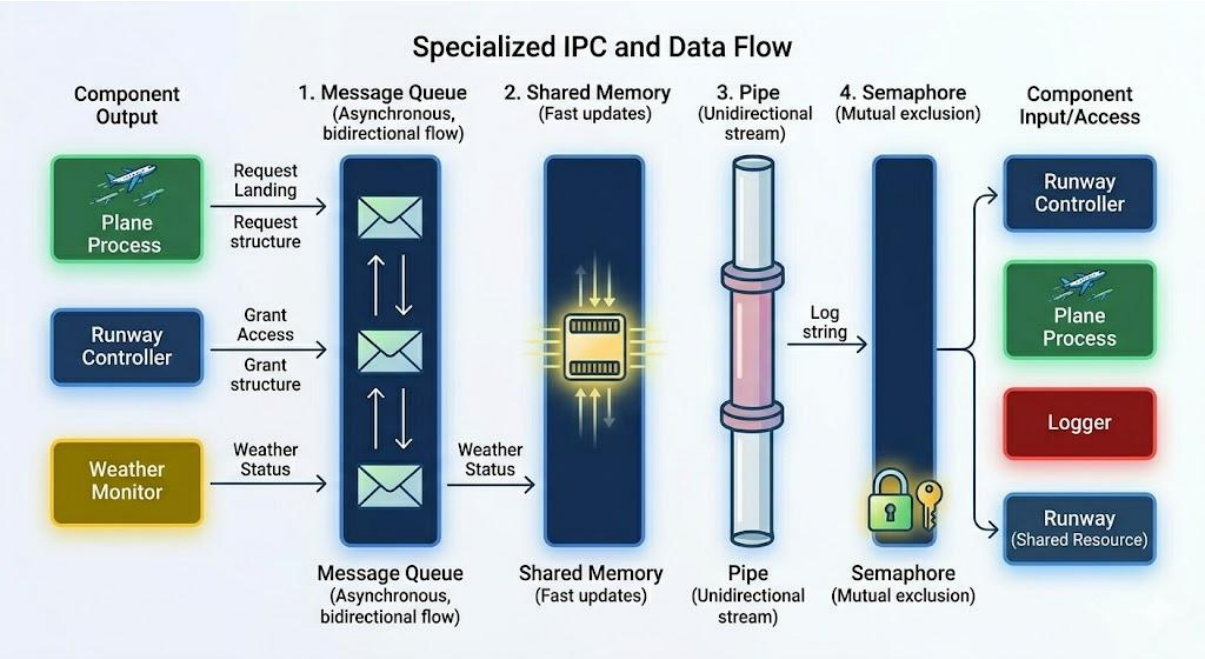
Plane Generator → Plane Processes → Runway Controller → Logger
↑
Weather Monitor

IPC Flow

Plane → Controller: Message Queue
Controller → Plane : Message Queue
Weather → Controller : Shared Memory
Controller → Logger : Pipe
Runway Access : Semaphore

Thread Flow (Controller)

```
Controller
  ├── Request Thread
  ├── Scheduler Thread
  └── Weather Thread
```



5. Process & Thread Implementation

Implementation Highlights

- fork() used for all processes
- exec() for modular execution
- waitpid() avoids zombies
- Threads created using pthread_create()
- Proper cleanup using exit() and signal handlers

Lifecycle Handling

- Parent monitors all children
- Threads safely synchronized
- Resources freed properly

6. IPC Usage & Correctness

IPC Mapping

Component	IPC Used	Reason
Plane → Controller	Message Queue	Asynchronous requests
Controller → Plane	Message Queue	Response
Weather → Controller	Shared Memory	Fast updates
Controller → Logger	Pipe	Stream logs
Runway Access	Semaphore	Mutual exclusion

7. IPC Implementation Quality

Features

- Blocking message queues → no busy waiting
- Shared memory for real-time updates
- Pipe ensures continuous logging
- Semaphore prevents runway conflict

Safety

- No deadlocks
- Controlled access to shared resources

- Proper synchronization

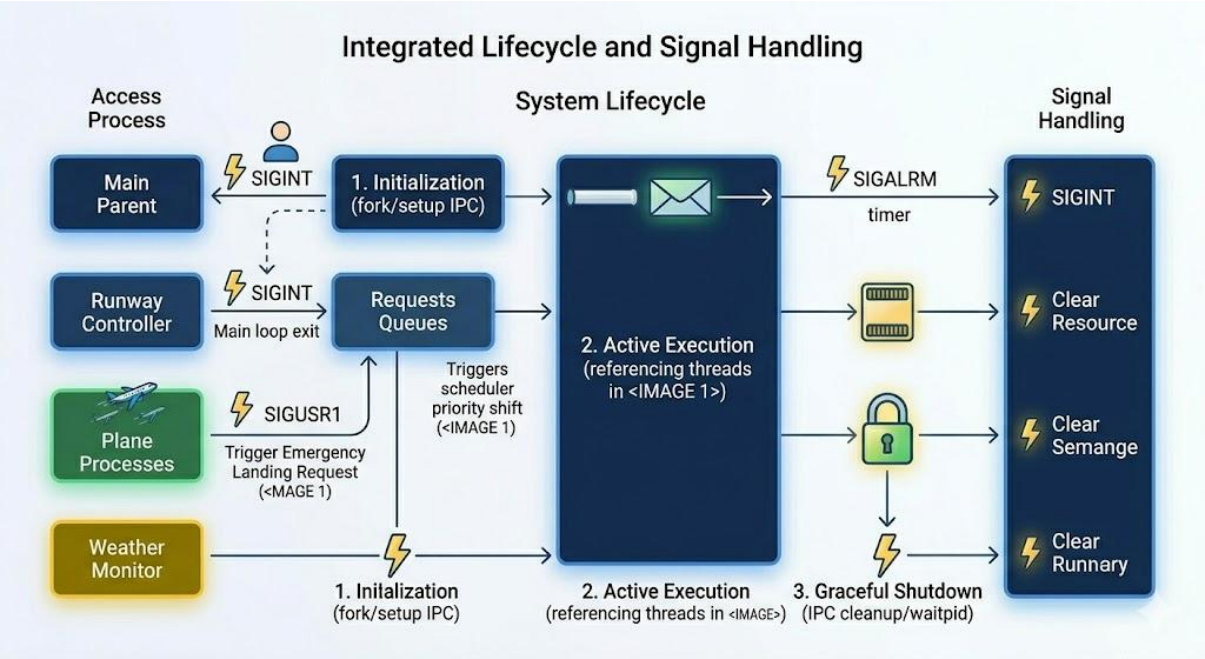
8. Signals & Lifecycle Management

Signals Used

Signal	Purpose
SIGINT	Graceful shutdown
SIGUSR1	Emergency landing
SIGALRM	Status updates

Behavior

- Emergency plane gets **highest priority**
- All processes terminate cleanly on SIGINT
- Periodic monitoring using alarms



9. File I/O & Data Management

Log File

System Calls Used

- open()
- write()
- lseek()
- close()

Example Logs

Plane 101 requested landing
Runway assigned to Plane 101
Weather Alert: Fog
Emergency landing for Plane 202

10. Code Quality & Engineering Practices

Practices Followed

- Modular structure (include/, src/)
- Header files for abstraction
- Error handling (perror)
- Clean naming conventions
- Separation of concerns

11. Makefile Correctness

Features

- Compiles all modules

- Supports run, clean, setup
- Uses variables for flexibility